

EX 8 – Demonstration of predictive models

AIM :

To demonstrate predictive models such as Decision Tree, Neural Network, and K-Nearest Neighbor (KNN) using datasets from various domains and evaluate their performance.

ALGORITHM :

1. Data Preparation

1. Import required libraries (Pandas, NumPy, Scikit-learn)
2. Load the dataset (e.g., healthcare, finance, or classification dataset)
3. Handle missing values and preprocess data
4. Split the dataset into **training** and **testing** sets

2. Decision Tree Model

1. Select features and target variable
2. Train the Decision Tree model using training data
3. Predict outcomes on test data
4. Evaluate using accuracy and confusion matrix

3. K-Nearest Neighbor (KNN)

1. Choose value of **K** (number of neighbors)
2. Train the KNN model
3. Predict class based on nearest neighbors
4. Evaluate model performance

4. Neural Network

1. Initialize the Neural Network model (e.g., Multi-Layer Perceptron)
2. Train the model on training data
3. Predict outputs using test data
4. Evaluate using accuracy and loss

5. Model Comparison

1. Compare performance of all three models
2. Identify the best model based on accuracy and results
3. Analyze strengths and limitations

CODE :

```
import warnings
```

```
warnings.filterwarnings("ignore")
```

```
import pandas as pd
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.tree import DecisionTreeClassifier
```

```
from sklearn.neighbors import KNeighborsClassifier
```

```
from sklearn.neural_network import MLPClassifier
```

```
from sklearn.metrics import accuracy_score
```

```
df = pd.read_csv("StudentsPerformance.csv")
```

```
df["performance"] = ((df["math score"] + df["reading score"] + df["writing score"]) / 3 > 70).astype(int)
```

```
X = df[["math score", "reading score", "writing score"]]
```

```
y = df["performance"]
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
```

```
dt = DecisionTreeClassifier()
```

```
dt.fit(X_train, y_train)
```

```
pred1 = dt.predict(X_test)
print(accuracy_score(y_test,pred1))
```

```
knn = KNeighborsClassifier(n_neighbors=3)
knn.fit(X_train,y_train)
pred2 = knn.predict(X_test)
print(accuracy_score(y_test,pred2))
```

```
nn = MLPClassifier(max_iter=1000)
nn.fit(X_train,y_train)
pred3 = nn.predict(X_test)
print(accuracy_score(y_test,pred3))
```

OUTPUT :

0.975

0.99

0.77

RESULT :

Thus the above program is executed and the result is obtained.